

Güvenlik Açığı Tarayıcısına Genel Bakış

[Kaynak](#)

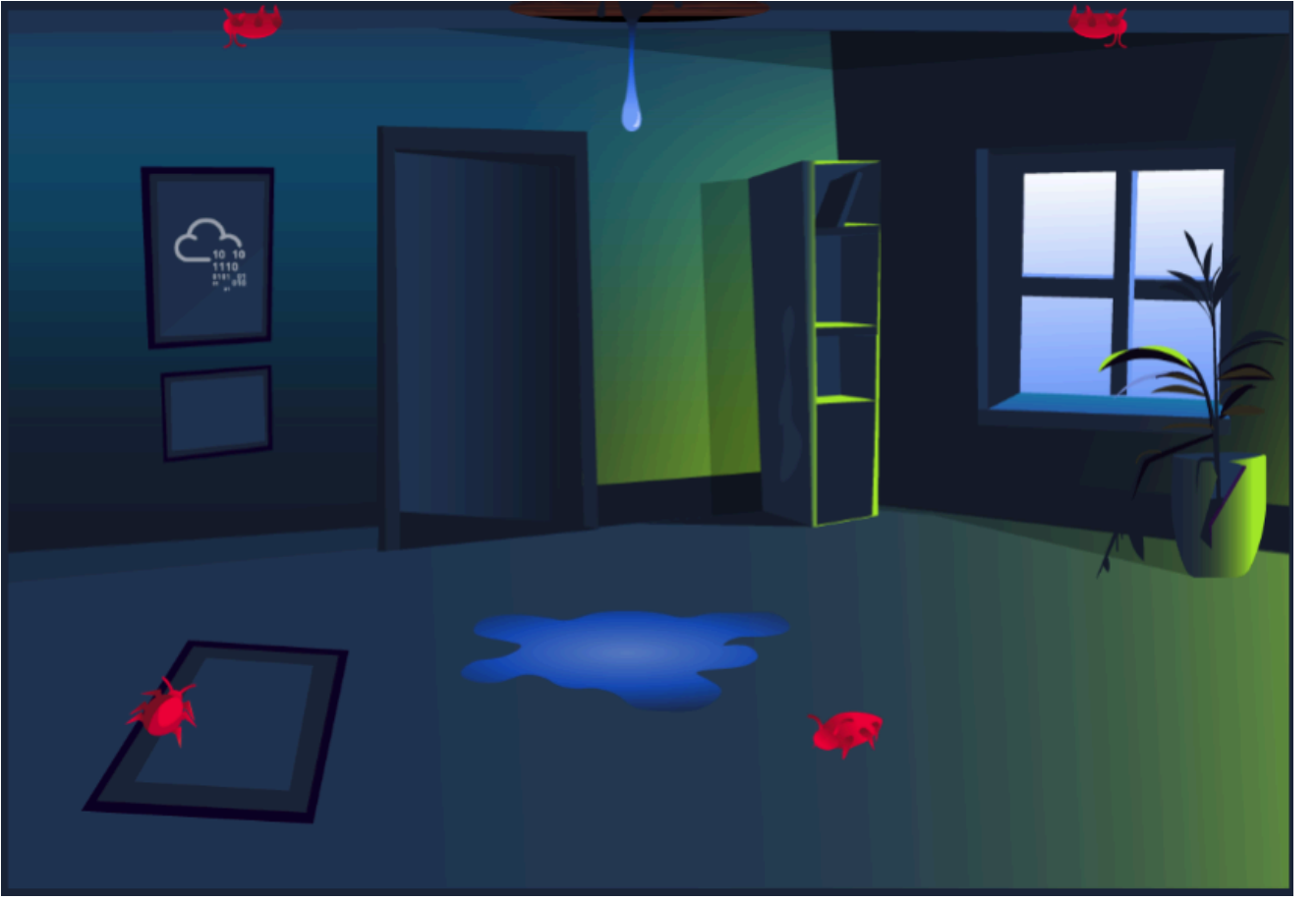
İÇERİK

1. Zafiyetler (Vulnerabilities) ve Yama Yönetimi
2. Zafiyet Taraması (Vulnerability Scanning) ve Analiz Metodolojisi
3. Zafiyet Tarama Ekosistemi: Sektörel Araçlar ve Raporlama Metodolojisi
4. CVE ve CVSS: Zafiyet Kimliklendirme ve Risk Skorlama Metodolojisi
5. OpenVAS: Konteyner Mimarisi Üzerinde Kurulum ve Operasyonel Tarama Metodolojisi

Zafiyetler (Vulnerabilities) ve Yama Yönetimi

Fiziksel Analojiden Dijital Gerçekliğe: Zafiyet Kavramı

Bir sistemin güvenliğini analitik düzeyde anlamak için öncelikle zayıflıkların doğasını kavramak gerekir. Bunu kavramsal olarak modellemek için fiziksel bir çatı analojisi kullanabiliriz. Çatısında küçük delikler bulunan bir ev düşünün; eğer bu delikler zamanında onarılmazsa, yağmur suyu, toz veya böcekler içeri sızarak evin iç yapısına ve eşyalara zarar verecektir. Evin bütünlüğünü tehdit eden bu yapısal zayıflıklar temel birer **Vulnerability** (Zafiyet - bir sistemin tasarımında, kodlanmasında veya yapılandırılmasında bulunan ve yetkisiz erişim/zararlı eylemler için istismar edilebilecek güvenlik açığı) örneğidir. Bu fiziksel zayıflıkları onararak evin güvenliğini yeniden sağlama işlemine ise **Patching** (Yama Uygulama - zafiyeti kapatmak amacıyla yazılıma veya sisteme uygulanan düzeltme, kod veya konfigürasyon güncellemesi) adı verilir.



Dijital Ekosistemde Zafiyetlerin Anatomisi

Fiziksel dünyadaki görünür deliklerin aksine, dijital cihazların yazılımlarında (işletim sistemleri, servisler, uygulamalar) veya donanımlarında (firmware, mimari kusurlar) bulunan zafiyetler çıplak gözle görülemez. Bu soyut ve görünmez doğa, dijital zafiyetleri çok daha kritik kılar. Saldırganlar (Threat Actors), sistemlere veya ağlara sızmak amacıyla sürekli olarak bu görünmez açıkları tarar ve **Exploit** (İstismar - bir zafiyeti kullanarak sisteme yetkisiz erişim sağlama veya zararlı komut çalıştırma yöntemi/kodu) ederek dijital cihazın kontrolünü ele geçirirler (Compromise).

Fiziksel bir çatıda oluşan hasar (su sızıntısı vb.) zamanla kendiliğinden fark edilebilirken, dijital cihazlardaki bir zafiyet aktif ve teknik bir incelemeyle aranmadığı sürece yıllarca sistemde sessizce barınabilir. Basit bir konfigürasyon hatası veya güncellenmemiş bir kütüphane gibi normal görünen bu zayıflıklar, zamanında tespit edilip yamanmazsa tüm ağın ele geçirilmesine veya devasa veri ihlallerine (Massive Damage) yol açar.

ÖNEMLİ: Dijital zafiyetler kendiliğinden ortaya çıkıp güvenlik ekiplerine uyarı vermezler. Siber güvenlikte temel gerçek, "senin proaktif olarak bulup kapatmadığın açığı, saldırganın er ya da geç bulacağı" prensibidir. Bu yüzden zafiyetleri tespit etmek, onlara adanmış özel bir efor, teknik derinlik ve sürekli bir "avlanma" (hunting) durumu gerektirir. Tespit sürecinin hemen ardından, zafiyetin kapatılarak sistemin güvence altına alındığı **Patching(Yama uygulama)** döngüsü başlatılmalıdır.

Zafiyet Avcılığı (Vulnerability Hunting) ve Otomasyonun Rolü

Bu çalışma modülünün temel odak noktası, dijital sistemlerin derinliklerinde gizlenen bu zafiyetleri proaktif bir şekilde avlamanın metodolojisini kavramaktır. Modern BT altyapılarının karmaşıklığı göz önüne alındığında, zafiyetleri tamamen manuel yöntemlerle aramak imkansızdır. Bu noktada süreci otomatize eden güvenlik araçları (Vulnerability Scanners) devreye girer.

İlerleyen pratik aşamalarda (Practical Demonstration), bu avlanma sürecini hızlandıran ve otomatize eden sektör standardı araçların çalışma mantığını inceleyeceğiz. Arka planda bu araçların sistemleri nasıl taradığını, bulguları nasıl anlamlandırdığını analiz edecek ve gerçek bir senaryo üzerinden bir aracın zafiyet tespitini nasıl gerçekleştirdiğini teknik adımlarıyla uygulayacağız.

Zafiyet Taraması (Vulnerability Scanning) ve Analiz Metodolojisi

Kurumsal Güvenlik Duruşu ve Uyumluluk (Compliance) İhtiyacı

Vulnerability Scanning (Zafiyet Taraması - dijital sistemlerin, ağların ve uygulamaların zayıflıklarını tespit etmek amacıyla yapılan sistematik denetim süreci), kurumların dijital altyapılarını güvence altına almalarının temel taşıdır. Saldırganlar, sistemleri **Compromise** (Ele geçirme/Sızma - bir sistemin güvenlik mekanizmalarını atlatarak yetkisiz erişim sağlama) etmek ve kritik verileri sızdırmak için sürekli olarak bu yapısal açıkları ararlar.

Bu taramalar sadece reaktif bir savunma mekanizması değil, aynı zamanda sektörel bir zorunluluktur. Birçok regülatör kurum ve güvenlik standardı (örneğin PCI-DSS, ISO 27001), altyapının durumuna göre üç ayda bir (quarterly) veya yılda bir kez (annually) düzenli zafiyet taramalarının yapılmasını bir **Compliance** (Uyumluluk - yasal ve sektörel güvenlik kurallarına uygunluk) şartı olarak koşar. Tespit edilen bulgular, yazılım veya sistem konfigürasyonlarında yapılan değişikliklerle, yani **Patches** (Yamalar) ile düzeltilerek sistem güvenli hale getirilir.

Manuel Zafiyet Avcılığının Sınırları ve Otomasyonun Gücü

Dijital altyapı (Network) büyüdükçe, zafiyetleri manuel olarak aramak imkansız, yorucu (hectic) ve hata yapmaya son derece açık bir sürece dönüşür. Manuel süreçte gözden kaçan tek bir kritik zafiyet, tüm ağın çökmesine neden olabilir. Bu darboğazı aşmak için sektörde otomatize edilmiş zafiyet tarama araçları (Vulnerability Scanners - örn: Nessus, OpenVAS, Qualys) kullanılır.

Bu araçların temel çalışma mantığı oldukça basittir ancak arka planda devasa bir otomasyon çalışır. Araca hedef olarak tek bir **IP adresi** (örn: 192.168.1.10) veya bir **Ağ Aralığı (Network Range)** (örn: 192.168.1.0/24) verilir. Araç, bu hedeflere binlerce farklı istek (probe) göndererek zafiyetleri otomatik olarak tespit eder ve okunabilir bir rapor sunar.

Aşağıda, karmaşık kurumsal araçların arka planındaki mantığı anlamak için, popüler ağ tarayıcısı Nmap'in kendi içindeki otomatize zafiyet tarama motorunu (Nmap Scripting

Engine) nasıl tetiklediğimize dair kavramsal bir örnek yer almaktadır:

```
nmap -sV --script vuln 192.168.1.50
```

Komutun ve Çıktının Teknik Analizi:

Burada Nmap aracına `-sV` parametresini vererek hedefteki (192.168.1.50) açık portlarda çalışan servislerin tam versiyonlarını (Service Versioning) tespit etmesini söylüyoruz. Hemen ardından `--script vuln` parametresiyle, Nmap'in veritabanındaki bilinen tüm otomatize zafiyet tarama betiklerini (scripts) bu tespit edilen versiyonlar üzerinde çalıştırmasını emrediyoruz. Eğer bu adım manuel yapılsaydı, her bir porttaki servis versiyonunu tek tek Google veya Exploit-DB üzerinden aratmamız gerekirdi; otomasyon bu süreci saniyeler içine sığdırır.

Tarama Derinliği: Authenticated vs. Unauthenticated Scans

Zafiyet taramaları, hedefe olan erişim yetkimize göre iki ana kategoriye ayrılır. Bu ayrım, saldırganın profilini (sistemde yetkisi olan biri mi yoksa dışarıdan saldıran rastgele biri mi?) simüle etmek için kritiktir.

ÖNEMLİ: Unauthenticated Scans (Kimlik Doğrulamasız Taramalar), hedef sistemin **Threat Surface** (Tehdit Yüzeyi - saldırganın sisteme sızmak için kullanabileceği tüm olası giriş noktalarının toplamı) analizini "dışarıdan bir gözle" (black-box) yapar. Araç, sisteme giriş yapacak (login) herhangi bir kimlik bilgisine (credentials) sahip değildir. Sadece dışarıya açık olan zafiyetleri (örn: herkesin erişebildiği bir web sunucusundaki SQL Injection açığı) tespit eder. Kurulumu basit ve sistem kaynaklarını daha az tüketen bir yöntemdir.

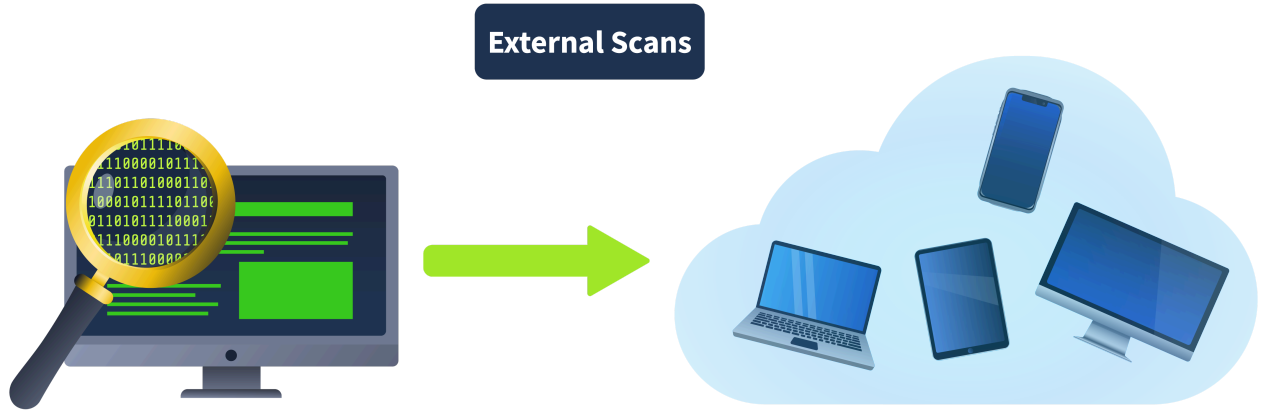
Buna karşın, **Authenticated Scans** (Kimlik Doğrulamalı Taramalar), araca hedef sistemin kimlik bilgilerinin (Kullanıcı adı/Parola, SSH anahtarları vb.) verilmesiyle yapılır. Araç (örneğin Nessus) sisteme yetkili bir kullanıcı gibi giriş yapar (log in). Bu sayede sadece dışarıdan görünen portları değil, işletim sisteminin derinliklerindeki konfigürasyon dosyalarını, yüklü olan uygulamanın spesifik yama sürümlerini (örneğin bir iç veritabanındaki eksik güncellemeleri) okuyabilir.

Özellik	Authenticated (Kimlik Doğrulamalı) Tarama	Unauthenticated (Kimlik Doğrulamasız) Tarama
Kimlik İhtiyacı	Zafiyet tarayıcısına hedef host'un kimlik bilgileri (Credentials) mutlaka verilmelidir.	Kimlik bilgisi gerekmez; tarayıcının sadece hedefin IP adresini bilmesi yeterlidir.
Tehdit Simülasyonu	Sisteme erişimi olan (içerideki) bir saldırganın/kullanıcının istismar edebileceği zafiyetleri tespit eder.	Hedef sisteme hiçbir erişimi olmayan, dışarıdaki bir saldırganın istismar edebileceği zafiyetleri tespit eder.

Özellik	Authenticated (Kimlik Doğrulamalı) Tarama	Unauthenticated (Kimlik Doğrulasız) Tarama
Görünürlük (Visibility)	Yüklü uygulamaları, yerel dosyaları ve konfigürasyonları tarayarak hedefe dair çok daha derin bir görünürlük sağlar.	Dışarıdan yapılan yüzeysel bir tarama olduğu için sistemin içine dair görünürlük sunmaz, sadece ağ seviyesindeki zafiyetleri yakalar.
Kullanım Senaryosu	İç ağda (Internal) bulunan ve dışarıya kapalı olan kritik bir veritabanı sunucusunun derinlemesine taranması.	İnternete açık (Public-facing) bir web uygulamasının, herhangi bir kullanıcının tetikleyebileceği açıklara karşı taranması.

Ağ Perspektifinden Taramalar: Internal vs. External Scans

Taramanın yapıldığı ağ konumuna (Network Topology) göre taramalar iki farklı savunma stratejisini temsil eder.



- **External Scans (Dış Taramalar):** Taramayı yapan cihaz, kurum ağının tamamen dışındadır (örneğin internet üzerinden tarama yapar). Amacı, kurumun dış dünyaya açılan kapılarındaki (Firewall, public web sunucuları, VPN gateway'leri) zafiyetleri bulmaktır. Dışarıdan gelen ilk saldırı dalgasında nelerin risk altında olduğunu gösterir.
- **Internal Scans (İç Taramalar):** Tarama, kurumun kendi ağının (LAN/VLAN) içinden yapılır. Bu yaklaşımın temel mantığı siber güvenlikteki "Assume Breach" (İhlal Edildiğini Varsay) prensibidir: "Eğer saldırgan bir şekilde güvenlik duvarını (perimeter) aşmış iç ağı sızarsa, içeride yatayda hareket (Lateral Movement) etmek için hangi zafiyetleri kullanabilir?" sorusuna cevap verir.

Parametre	Internal Scans (İç Taramalar)	External Scans (Dış Taramalar)
Tarama Konumu	Kurumun iç ağından (Internal Network) dışarıya veya içerideki	Ağın dışından (İnternet/WAN üzerinden) kurumun dışarıya açık

Parametre	Internal Scans (İç Taramalar)	External Scans (Dış Taramalar)
	diğer cihazlara doğru yapılır.	yüzüne doğru yapılır.
Odak Noktası	Yalnızca ağın içine girildiğinde istismar edilebilecek zafiyetleri tespit etmeye odaklanır.	Dışarıdan (internet üzerinden) doğrudan istismar edilebilecek zafiyetlere odaklanır.

DİKKAT (Stratejik Kural): Bu tarama türleri genellikle birbirini tamamlar nitelikte eşleştirilir. Sektörel en iyi pratiklerde (Best Practices), iç ağdaki (Internal) cihazların derinlemesine güvenlik durumu için **Authenticated Scans** tercih edilirken; dış dünyaya açık (External) sistemlerin internetteki bir saldırganın gözünden nasıl görüldüğünü anlamak için **Unauthenticated Scans** uygulanır. Gerçek bir güvenlik duruşu, tüm bu yöntemlerin birlikte ve düzenli kullanılmasıyla elde edilir.

Zafiyet Tarama Ekosistemi: Sektörel Araçlar ve Raporlama Metodolojisi

Endüstri Standartlarında Zafiyet Tarama Araçları (Vulnerability Scanners)

Siber güvenlik operasyonlarında, hedefin büyüklüğüne, bütçeye ve mimari gereksinimlere (bulut, yerel ağ, hibrit) göre doğru aracın seçilmesi kritik bir stratejik karardır. Piyasada bulunan otomatize tarayıcıların her biri farklı bir felsefeyle (deployment model, risk hesaplama, varlık yönetimi) çalışır. Aşağıda sektörde en çok karşılaşılan ve mimarileri gereği farklı kullanım senaryolarına hitap eden dört temel aracı analiz ediyoruz.

Nessus (Tenable): Kurumsal Standart ve On-Premises Güç

1998 yılında açık kaynaklı (open-source) bir proje olarak başlayan Nessus, 2005 yılında Tenable tarafından satın alınarak tescilli (proprietary) bir yazılıma dönüştürülmüştür. Günümüzde büyük ölçekli kurumsal ağlarda (Large Enterprises) fiili standart (de facto) olarak kabul edilir.

- Dağıtım Modeli (Deployment):** Nessus, doğrudan kurumun kendi sunucularına veya fiziksel donanımlarına kurularak yönetilmelidir (**On-premises**). Bu durum, dışarıya veri çıkartmak istemeyen yüksek güvenlikli ortamlar için idealdir.
- Lisanslama ve Kapsam:** Ücretsiz versiyonu (Nessus Essentials) ağdaki tarama sayısını ve özelliklerini kısıtlarken; ticari versiyonu (Nessus Professional) gelişmiş tarama profilleri, sınırsız hedef IP desteği ve profesyonel destek sunar.

Qualys: Bulut Tabanlı Sürekli İzleme (Continuous Monitoring)

1999 yılında piyasaya sürülen Qualys, geleneksel tarayıcılardan farklı olarak tamamen bulut tabanlı (Cloud-based platform) bir **SaaS (Software as a Service)** olarak tasarlanmıştır.

- **Mimari Avantajı:** Sistem fiziksel bir donanıma kurulmadığı için bakım, güncelleme veya donanım maliyeti gerektirmez. Kurum ağına yerleştirilen küçük sensörler (appliances) üzerinden veriler doğrudan Qualys bulutuna aktarılır.
- **Odak Noktası:** Yalnızca belirli zamanlarda yapılan taramaların ötesine geçerek ağdaki varlıkları (Asset Management) sürekli izler ve yeni bir zafiyet/uyumsuzluk (Compliance Check) tespit ettiğinde anlık uyarı (alert) üretir.

Nexpose (Rapid7): Risk Odaklı Varlık Yönetimi

2005 yılında Rapid7 tarafından geliştirilen Nexpose, zafiyetleri sadece tespit etmekle kalmaz, onları kurumun iş süreçlerine verebileceği zarara göre önceliklendirir.

- **Risk Skoruması (Risk Scoring):** Nexpose'un en büyük teknik farkı, bir zafiyetin risk puanını hesaplarken sadece açığın teknik şiddetini (CVSS) değil, aynı zamanda açığın bulunduğu **Varlığın Değerini (Asset Value)** ve saldırının **Etkisini (Impact)** hesaba katmasıdır. (Örn: Veritabanı sunucusundaki düşük seviyeli bir açık, test sunucusundaki kritik bir açıktan daha yüksek puanlanabilir).
- **Esnek Dağıtım:** Hem **On-premises** hem de **Hybrid** (Bulut ve yerel sistemlerin entegre çalıştığı) modları destekleyerek mimari esneklik sağlar.

OpenVAS: Açık Kaynaklı Temel Güvenlik Değerlendirmesi

Greenbone Security tarafından geliştirilen **OpenVAS (Open Vulnerability Assessment System)**, ticari rakiplerine kıyasla tamamen açık kaynaklı ve ücretsiz bir alternatiftir.

- **Kullanım Senaryosu:** Kapsamı ticari araçlar kadar devasa olmasa da, kendi veritabanındaki bilinen zafiyetleri (Known Vulnerabilities) başarıyla tespit edebilir. KOBİ'ler (Small Organizations) veya sistemlerini bireysel olarak test etmek isteyen güvenlik uzmanları için mükemmel bir giriş noktasıdır.

Araçların Karar Mantığı ve Raporlama (Reporting Capabilities)

Bir zafiyet tarama aracını çalıştırmanın arka planındaki temel mantık, ağdaki zayıflıkları tespit edip düzeltme adımlarını (remediation) belirlemektir. Detaylı teknik adımları ve karmaşık istismar senaryolarını atlayarak süreci özetlemek gerekirse: Araç, hedef sistemdeki servisleri tarar, bulgularını kendi veritabanıyla eşleştirir ve tespit edilen açıklar için en doğrudan yama veya yapılandırma önerilerini sunar. Bu yüksek seviyeli tespit ve eşleştirme mantığı, kurumların risklerini önceliklendirmesini sağlar.

Taramalar sonucunda elde edilen verilerin kalbi **Raporlama (Reporting)** aşamasıdır.

[VULNERABILITY DETECTED]

Target: 192.168.10.55

Asset Value: High (Production Database)

Plugin ID: 104743 | Severity: CRITICAL | Risk Score: 9.8

Description: The remote host is missing a security update for SMB (MS17-

010).

Remediation: Apply the appropriate Microsoft patch (KB4013389) or disable SMBv1.

Çıktının Teknik Analizi: Yukarıdaki kavramsal log çıktısı, gelişmiş bir tarayıcının (örn: Nexpose veya Nessus) nasıl bir sonuç üreteceğini göstermektedir. Çıktı sadece zafiyetin adını (MS17-010) vermekle kalmaz, aynı zamanda sistemin kritiklik seviyesini (Asset Value: High) ve nasıl çözüleceğini (**Remediation**) net bir şekilde belirtir. Siber güvenlikte raporlar sadece sorunları değil, çözüm yollarını (Remediation Methods) sunarak IT/Sistem ekiplerinin doğrudan aksiyon almasını sağlamalıdır.

ÖNEMLİ: Dijital varlıklarınız için doğru aracı seçerken ağın kapsamı (Scope), eldeki sistem/donanım kaynakları (Resources) ve ihtiyaç duyulan analizin derinliği (Depth of Analysis) belirleyici faktörlerdir. Hiçbir araç tek başına %100 güvenlik sağlamaz; uzmanlık, bu raporlardaki teknik verileri doğru okuyup, riskleri kurumun mimarisine göre yorumlayabilmektir.

CVE ve CVSS: Zafiyet Kimliklendirme ve Risk Skorlama Metodolojisi

Zafiyet Yönetiminde Standardizasyon İhtiyacı

Büyük ölçekli bir IT altyapısını veya bir Siber Güvenlik Operasyon Merkezi'ni (SOC) yönettiğinizi düşünün. Her gün yüzlerce yeni sistem hatası, ağ kesintisi veya güvenlik uyarısıyla karşılaşsınız. Bu kaos ortamında hangi güvenlik açığının hangi sisteme ait olduğunu, hangisinin daha acil yama gerektirdiğini takip etmek manuel yöntemlerle imkansızdır. Siber güvenlik endüstrisinde bu karmaşayı çözmek, dünya çapında bir dil birliği sağlamak ve zafiyetleri evrensel bir standartta takip edebilmek için **CVE** ve **CVSS** yapıları kullanılır.

CVE (Common Vulnerabilities and Exposures)

CVE (Ortak Zafiyetler ve Etkilenmeler), tespit edilen her bir spesifik güvenlik açığına atanan evrensel ve benzersiz bir kimlik numarasıdır. Bir nevi dijital dünyadaki hastalıkların uluslararası teşhis kodları gibi çalışır. **MITRE** Corporation tarafından geliştirilen ve yönetilen bu sistem, herhangi bir yazılımda, donanımda veya mimaride yeni bir zafiyet keşfedildiğinde devreye girer.

Amaç, güvenlik açıklarını küresel bir veritabanında (CVE Database) yayımlayarak güvenlik uzmanlarının, sistem yöneticilerinin ve araçların (Vulnerability Scanners) aynı zafiyetten bahsettiğinden emin olmaktır. Bu sayede bir zafiyet tespit edildiğinde, herkes o zafiyetin detaylarına, nasıl istismar edileceğine ve en önemlisi onu düzeltmek için (Remediation) hangi adımların atılması gerektiğine tek bir referans numarası üzerinden ulaşabilir.

Aşağıda, bir sistemde bulunan zafiyetin otomatize bir araç veya bir API sorgusu üzerinden bize nasıl döndüğünü gösteren kavramsal bir çıktı yer almaktadır:

```
{  
  "Vulnerability_ID": "CVE-2024-9374",  
  "Assigner": "MITRE Corporation",  
  "Status": "Published",  
  "Description": "Unauthenticated Remote Code Execution in XYZ Web Server.",  
  "Remediation": "Upgrade to version 2.1.5 immediately."  
}
```

Çıktının ve İsimlendirme Formatının Teknik Analizi:

Yukarıdaki veri bloğunda gördüğümüz CVE-2024-9374 numarası rastgele verilmemiştir; uluslararası standartlara göre üç ana parçadan oluşur:

- CVE Prefix:** Her kayıt zorunlu olarak "CVE" ön ekiyle başlar; bu, numaranın bağlamını belirler.>
- Year (Yıl):** Ortadaki bölüm (2024), zafiyetin keşfedildiği veya halka açıklandığı yılı temsil eder. Bu, eski ve yeni zafiyetleri ayırt etmek için kritik bir zaman damgasıdır.
- Arbitrary Digits (Sıra Numarası):** Son bölüm (9374), o yıl içerisinde keşfedilen zafiyetlere sırayla atanan en az dört haneli bir sayıdır. Sadece bu satıra bakarak, bu zafiyetin 2024 yılında kaydedilmiş, kimlik doğrulaması gerektirmeyen (Unauthenticated) kritik bir uzaktan kod çalıştırma (RCE) açığı olduğunu ve acilen sürüm yükseltmesi (Upgrade) yapılması gerektiğini anlıyoruz.

CVSS (Common Vulnerability Scoring System)

CVE numaraları bize "hangi" zafiyetle karşı karşıya olduğumuzu söyler, ancak "ne kadar tehlikeli" olduğunu veya yama işlemine "nereden başlamamız gerektiğini" söylemez. İşte bu noktada **CVSS** (Ortak Zafiyet Puanlama Sistemi) devreye girer.

Eğer sisteminizde aynı anda 50 farklı CVE tespit edildiyse, kısıtlı zaman ve kaynaklarla hangisini önce kapatacaksınız? CVSS, bu önceliklendirme (Prioritization) problemini çözmek için her bir zafiyete 0.0 ile 10.0 arasında matematiksel bir risk skoru atar. Bu skor rastgele verilmez; arka planda zafiyetin **Impact** (Sisteme etkisi - Gizlilik, Bütünlük ve Erişilebilirlik kaybı) ve **Exploitability** (İstismar edilebilirliği - Saldırganın bu açığı kullanmak için ağ erişimine mi yoksa sadece internete mi ihtiyacı olduğu, sistemin karmaşıklığı vb.) gibi birçok teknik faktörün hesaplandığı bir algoritma çalışır.

ÖNEMLİ: Skor ne kadar yüksekse, zafiyetin kurumsal altyapıya verebileceği zarar ve saldırganlar tarafından kullanılma kolaylığı o kadar yüksektir. Güvenlik ekipleri (Blue Team), yama yönetimi süreçlerini bu skorların belirlediği aciliyet seviyelerine göre yönetir.

Sektörde geçerli olan CVSS v3.x standartlarına göre şiddet (Severity) sınıflandırması şu şekildedir:

CVSS Skor Aralığı	Şiddet Seviyesi (Severity Level)	Operasyonel Anlamı ve Müdahale
0.0 - 3.9	Low (Düşük)	Genellikle bilgi sızdırması veya istismarı çok zor olan yapılandırma hatalarıdır. Acil müdahale gerektirmez, rutin bakımlarda yamalanır.
4.0 - 6.9	Medium (Orta)	Kısıtlı etkiye sahip veya istismarı için yetkili bir kullanıcı profili gerektiren zafiyetlerdir. Planlı yama döngülerine dahil edilir.
7.0 - 8.9	High (Yüksek)	Çoğunlukla uzaktan istismar edilebilen ve veri sızıntısına/kısmi sistem ele geçirmeye yol açan açıklardır. Kısa sürede kapatılmalıdır.
9.0 - 10.0	Critical (Kritik)	DİKKAT: Saldırganın internet üzerinden, hiçbir kimlik doğrulamasına ihtiyaç duymadan (Unauthenticated) doğrudan sistemi tamamen ele geçirmesine (Root/System yetkisi) olanak tanıyan zafiyetlerdir. Hiçbir planlı döngü beklenmez, sistem gerekirse anında kapatılıp (Downtime göze alınarak) acil yama (Emergency Patch) uygulanmalıdır.

OpenVAS: Konteyner Mimarisi Üzerinde Kurulum ve Operasyonel Tarama Metodolojisi

Bağımlılık Cehennemi (Dependency Hell) ve Konteynerizasyon Çözümü

Önceki analizlerimizde OpenVAS'ın (Open Vulnerability Assessment System) açık kaynaklı ve tam donanımlı bir zafiyet tarayıcısı olduğunu belirtmiştik. Ancak OpenVAS'ın doğası gereği, tarama motoru (scanner), veritabanı (Redis/PostgreSQL), yönetim arka planı (manager daemon) ve web arayüzü (Greenbone Security Assistant) gibi birçok farklı bileşeni vardır. Bu mimariyi doğrudan bir Linux ana makinesine (Host) kurmaya kalktığınızda, onlarca farklı kütüphane ve versiyon uyumsuzluğuyla (Dependency Hell - Bağımlılık Cehennemi) karşılaşsınız. Kurulum süreci oldukça yorucu (hectic) ve hata yapmaya açıktır.

Bu operasyonel darboğazı aşmak ve izole, standartlaştırılmış bir ortam yaratmak için sektörde standart haline gelen **Docker** teknolojisini kullanıyoruz. Docker, uygulamaları çalışması için gereken tüm sistem kütüphaneleri, konfigürasyon dosyaları ve bağımlılıklarıyla birlikte tek bir **Container** (Konteyner - uygulamanın ana işletim sisteminden izole bir şekilde, kendi sanal ortamında çalışmasını sağlayan paket) içine hapseder. Bu sayede OpenVAS'ı sıfırdan derlemek yerine, her şeyin önceden kurulup ayarlandığı hazır bir imajı saniyeler içinde ayağa kaldırabiliriz.

Docker Altyapısının Hazırlanması ve OpenVAS'ın Ayağa Kaldırılması

Ubuntu tabanlı bir sistemde zafiyet tarama operasyonumuza başlamak için öncelikle altyapımızda konteyner motorunu aktif hale getirmeliyiz.

Bash

```
sudo apt install docker.io
```

Komutun Analizi: Bu komut, Debian/Ubuntu sistemlerinin resmi paket depolarından Docker motorunu sisteme kurar. `sudo` kullanarak root yetkileriyle kurulumu gerçekleştiriyoruz çünkü ağ arayüzlerine ve kernel seviyesindeki izolasyon mekanizmalarına müdahale edilecek. **B Planı:** Eğer bu adımda "paket bulunamadı" hatası alırsanız, depolarınız güncel değildir. Öncesinde `sudo apt update` çalıştırmanız gerekir.

Docker motoru aktif edildikten sonra, karmaşık OpenVAS mimarisini tek bir komutla ayağa kaldırıyoruz. Operasyonel basitlik ve laboratuvar ortamı için tüm bileşenleri tek bir imajda toplayan `immauss/openvas` imajını tercih ediyoruz.

Bash

```
sudo docker run -d -p 443:443 --name openvas immauss/openvas
```

Parametre Analizi ve Teknik Mantık:

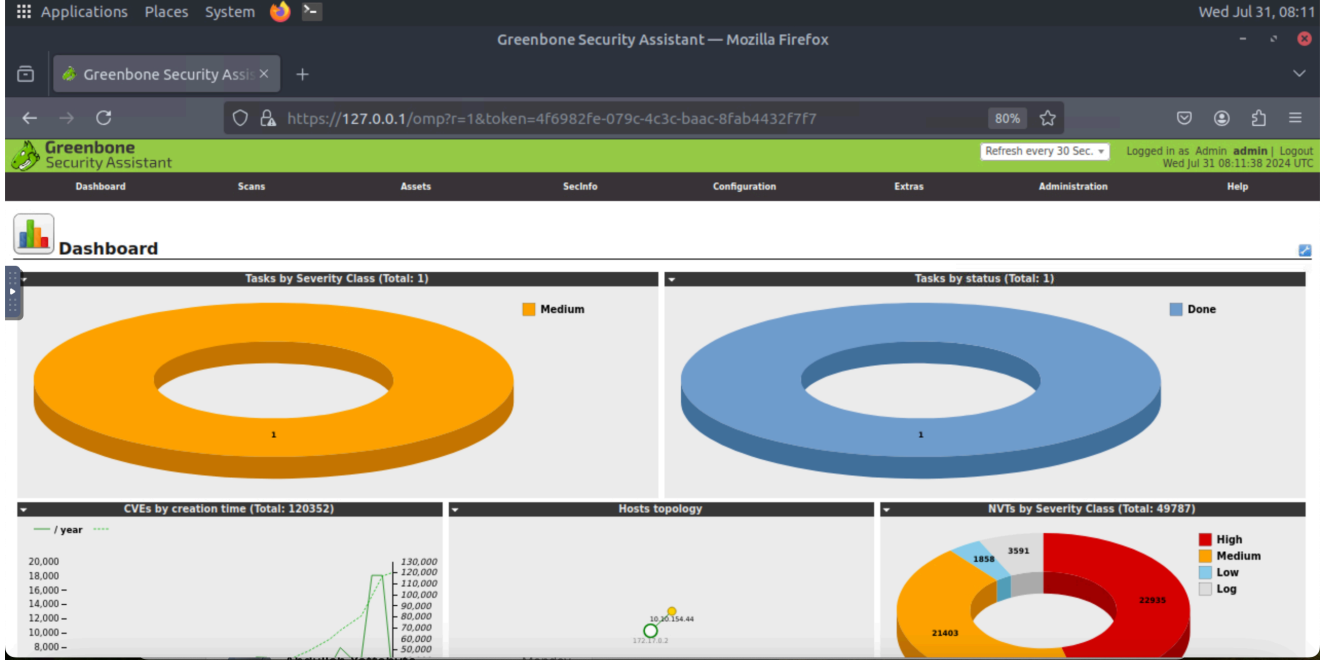
- `run` : Docker motoruna belirtilen imajdan yeni bir konteyner oluşturmasını ve çalıştırmasını söyler.
- `-d` (Detached Mode): Konteynerin arka planda (background) çalışmasını sağlar. Eğer bu parametreyi vermeseydik, terminalimiz konteynerin loglarına kilitlenir ve terminali kapattığımızda OpenVAS da kapanırdı.
- `-p 443:443` (Port Mapping): Burası kritik. Sol taraf Host (bizim makinemiz), sağ taraf Konteynerin içidir. OpenVAS arayüzü içeride 443 numaralı HTTPS portundan yayın yapmaktadır. Bu parametre ile konteynerin 443 portunu, kendi makinemizin 443 portuna bağlıyoruz.
- `--name openvas` : Oluşturulan konteynere rastgele bir isim atanması yerine, yönetimi kolaylaştırmak için "openvas" ismini veriyoruz. İleride `docker stop openvas` diyerek sistemi kolayca durdurabiliriz.
- `immauss/openvas` : Docker Hub üzerinden çekilip çalıştırılacak hedef imajın adıdır. Eğer imaj lokalde yoksa, otomatik olarak internetten indirilecektir (pull).

Alternatif Senaryo (Port Çakışması): Eğer host makinanızda zaten 443 portunu kullanan başka bir web sunucusu (örn: Apache/Nginx) varsa bu komut başarısız olur. Bu durumda `-p 8443:443` parametresiyle port yönlendirmesini değiştirerek çakışmayı önleyebilirsiniz.

Yönetim Paneline (Web Interface) Erişim ve Güvenlik Bağlamı

Konteyner başarıyla çalışmaya başladıktan sonra, OpenVAS yönetim arayüzüne tarayıcınız üzerinden erişebilirsiniz. Sistem HTTPS üzerinden yayın yaptığı için doğrudan lokal ağ arayüzünüze (Loopback Interface) gitmeniz gerekir.

Erişim Adresi: <https://127.0.0.1>



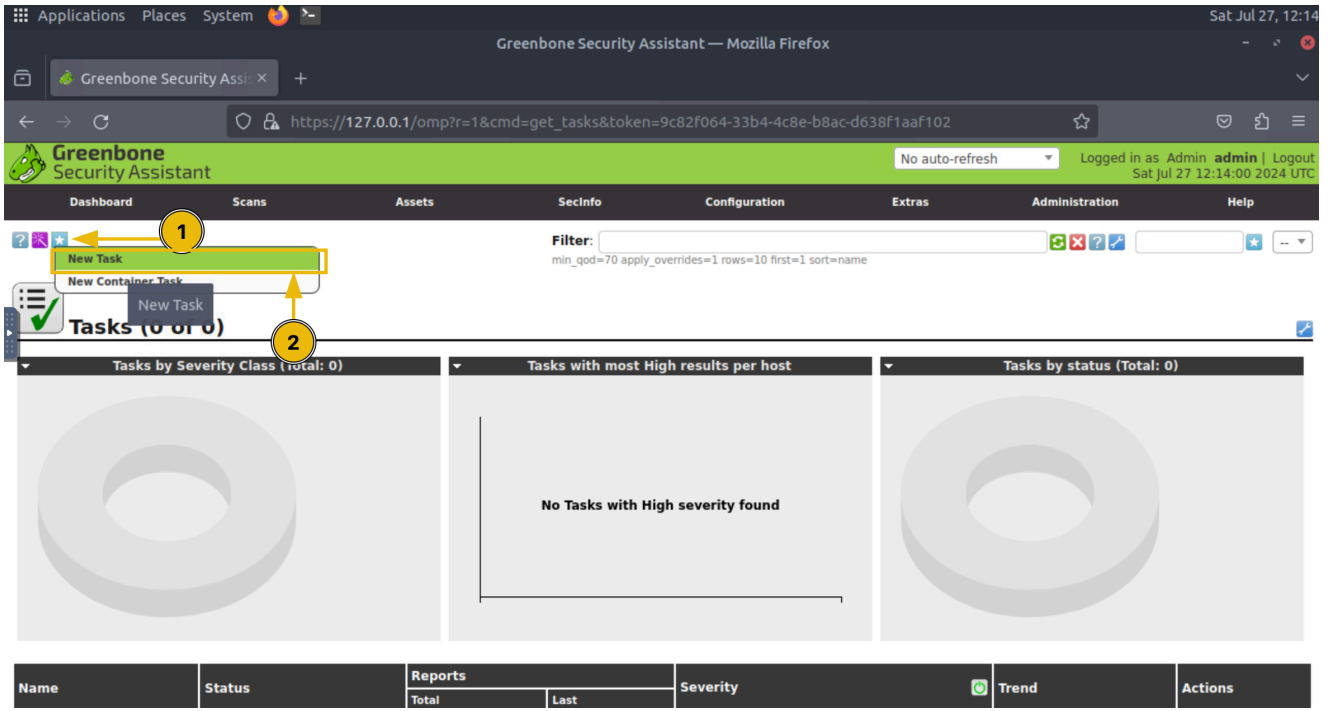
DİKKAT: Web tarayıcınıza bu adresi girdiğinizde büyük ihtimalle "Bağlantınız güvenli değil" veya "Geçersiz Sertifika" uyarıları alacaksınız. Bu durum operasyonel bir hata değildir. OpenVAS, kurulum sırasında kendi kendini imzalayan bir sertifika (Self-Signed Certificate) üretir ve tarayıcınız bu sertifikayı üreten otoriteyi (CA) tanımadığı için bu uyarıyı verir. Bir güvenlik uzmanı olarak arka plandaki bu mantığı bilmeli ve riski kabul ederek (Accept the risk and continue) login ekranına geçmelisiniz.

Doğru kimlik bilgileriyle (Credentials) giriş yaptıktan sonra, tüm zafiyet tarama operasyonlarınızı yöneteceğiniz **Dashboard** (Gösterge Paneli) karşınıza çıkar.

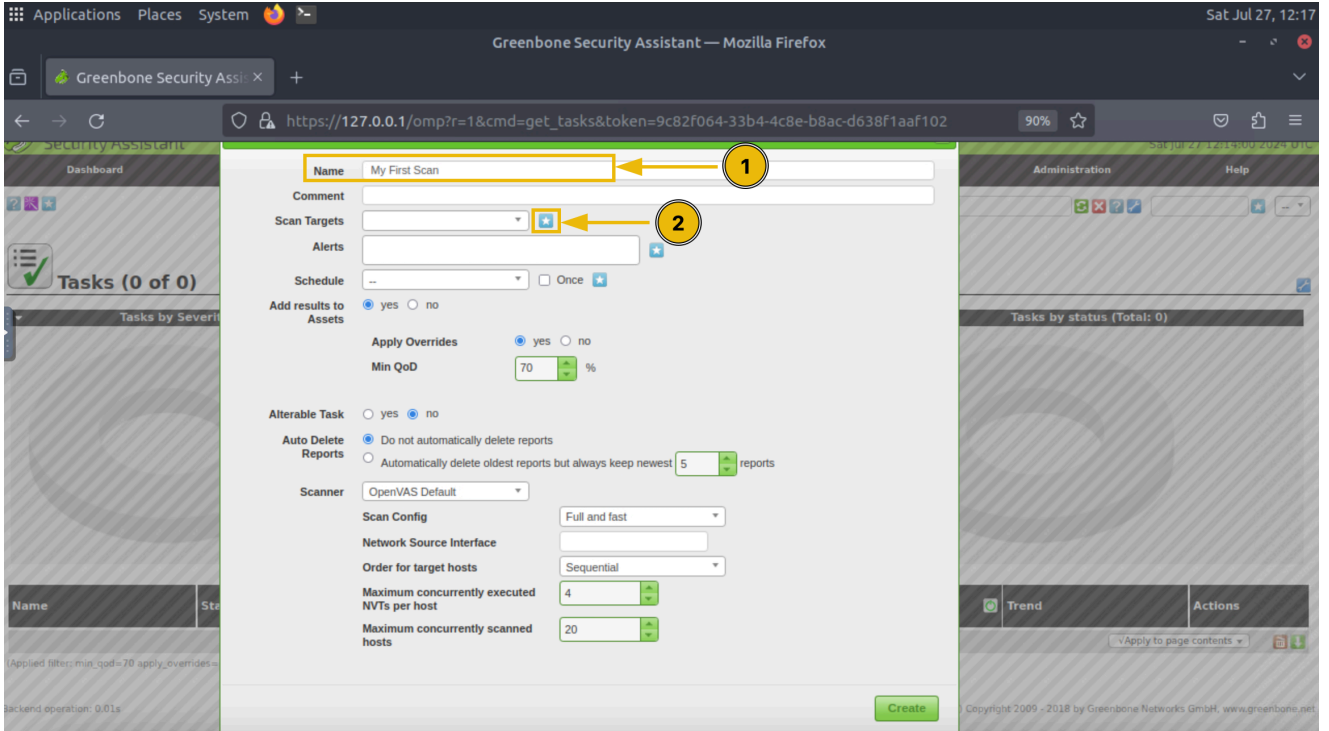
Hedef Belirleme ve Tarama (Vulnerability Scan) Operasyonu

OpenVAS üzerinde bir tarama başlatmak, birbirine bağlı mantıksal yapılandırmalar (Target, Task, Configuration) zincirinden oluşur. Sistemin sadece bir "IP yazıp tara" aracından öte kurumsal bir altyapı olduğunu unutmayın. Operasyonel akış şu şekildedir:

- 1. Task (Görev) Oluşturma:** İlk adımda doğrudan hedefi belirtemeyiz; önce bir "Görev" iskeleti kurmalıyız. Dashboard üzerindeki Scans menüsünden Tasks sekmesine gidiyoruz. Bu sayfa, geçmişteki ve devam eden tüm taramaların komuta merkezidir. Sol üst köşedeki yıldız (Star) ikonuna tıklayarak New Task seçeneğiyle yeni bir tarama görevi başlatıyoruz. Göreve, taramanın amacını belli eden kurumsal bir isim (örn: "DMZ-Web-Server-Scan-Q1") verilmelidir.

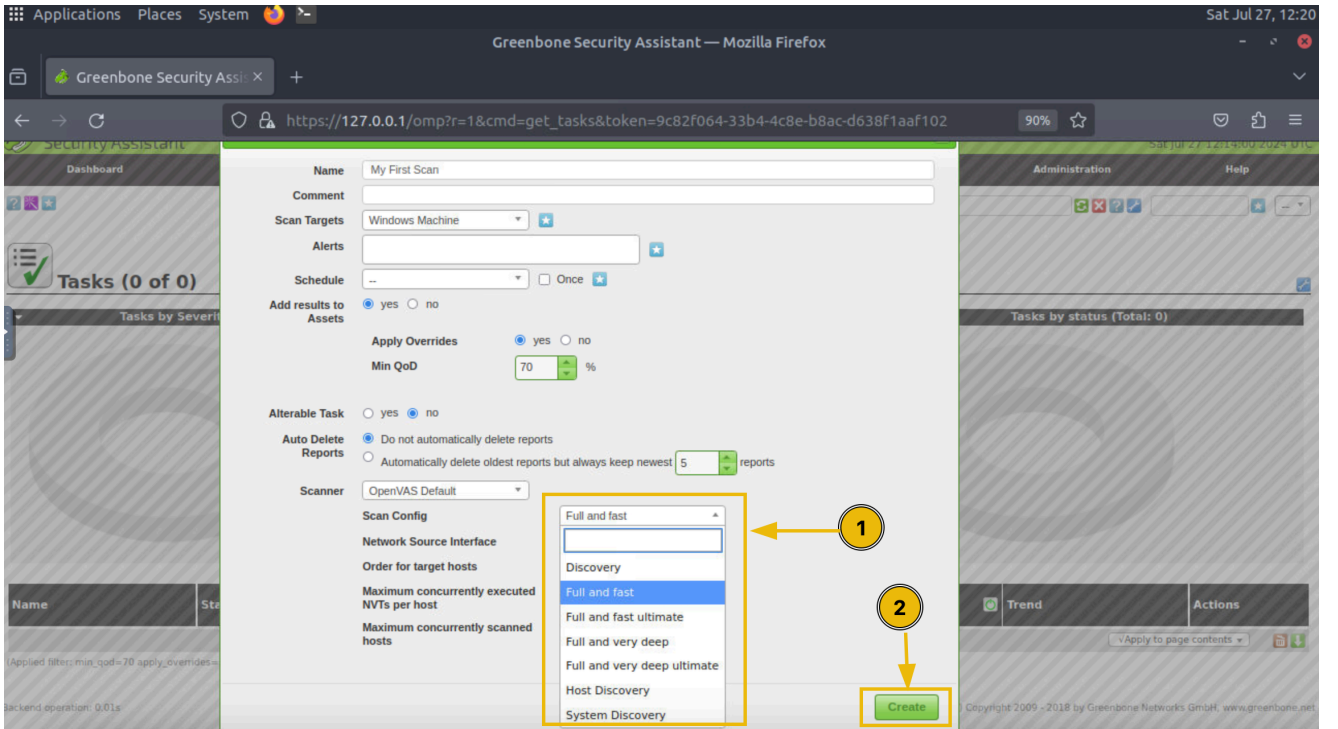
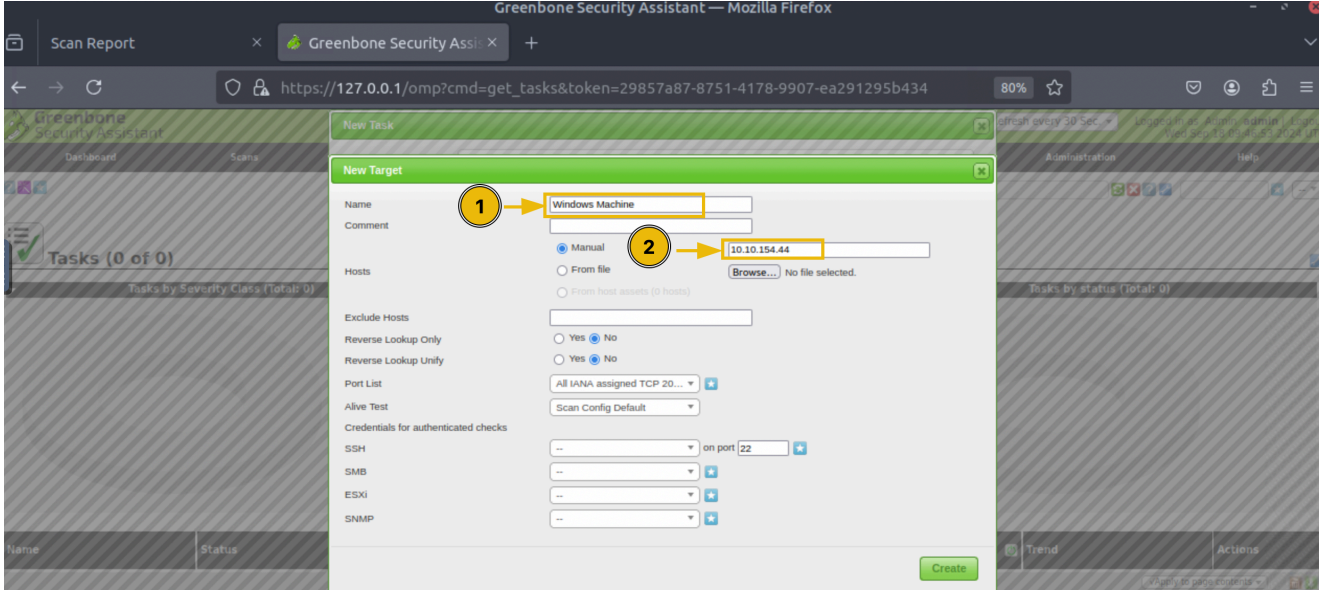


2. **Scan Targets (Tarama Hedefleri) Tanımlama:** Görev penceresi içerisinde hedefi tanımlamak için Scan Targets yanındaki ikonlara tıklıyoruz. Burada sisteme taranacak varlığı tanıtlıyoruz. Taranacak makinenin adını ve en önemlisi **IP Adresini** giriyoruz. Bu aşamada, eğer kimlik doğrulamalı (Authenticated) bir tarama yapacaksak (önceki notlarda bahsettiğimiz üzere), sistemin SSH/SMB kimlik bilgilerini de bu hedefe bağlamamız gerekir.



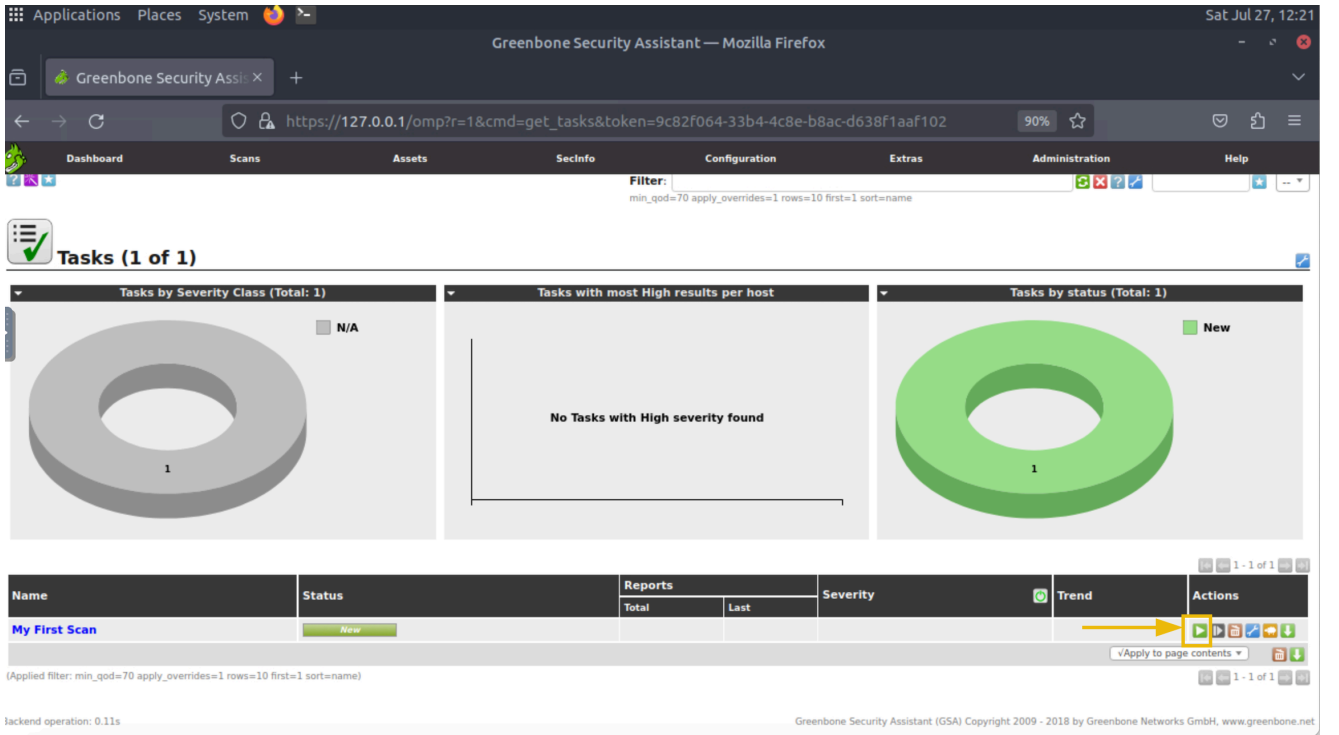
3. **Kapsam (Scope) ve Tarama Profilinin Seçimi:** Hedef belirlendikten ve oluşturulduktan (Create butonu) sonra, araca "nasıl" bir tarama yapması gerektiğini söylemeliyiz. OpenVAS farklı derinlik seviyelerine sahip birçok tarama seçeneği (Scan Config) sunar. Hızlı bir tarama mı (Discovery), yoksa tüm TCP/UDP portlarının ve derin konfigürasyon

dosyalarının incelendiği tam kapsamlı (Full and fast/Deep) bir tarama mı yapılacağına operasyonun ihtiyacına göre karar verilir.

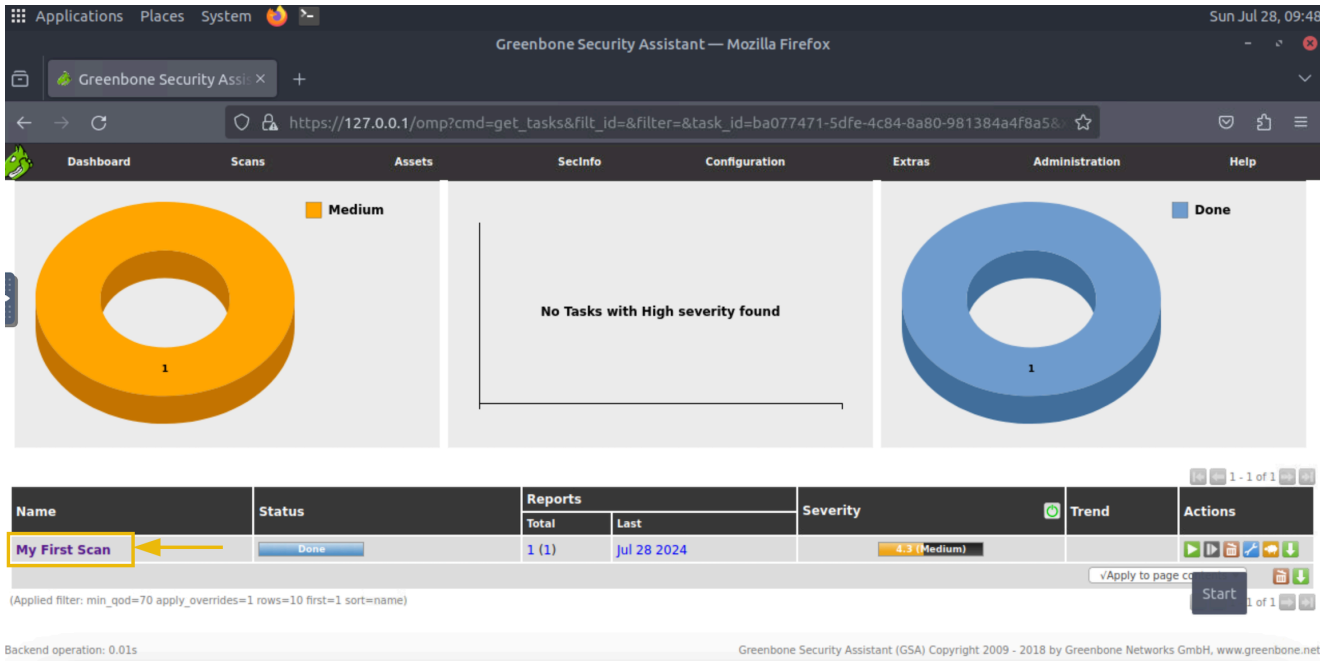


Analiz, Raporlama ve Remediation (Çözüm) Aşaması

Görev yaratıldıktan sonra Tasks paneline düşer. Tarama işlemini başlatmak için görevin Actions (Eylemler) sütunundaki Play (Başlat) butonuna tıklanır. Bu aşamada OpenVAS, hedefe yönelik binlerce script ve paket göndererek NVTs (Network Vulnerability Tests - Ağ Zafiyet Testleri) veritabanındaki imzalarla hedefin yanıtlarını karşılaştırmaya başlar.



Tarama süresi ağın hızına ve seçilen derinliğe göre değişir. Statü Done (Tamamlandı) olarak güncellendiğinde, sistem bize bulguların bir özetini grafiklerle sunar.



Applications Places System Greenbone Security Assistant — Mozilla Firefox

Greenbone Security Assi: x +

https://127.0.0.1/omp?cmd=get_task&task_id=dc4db8a9-f71e-4b55-8aee-a2cf4247da1e&overrides= 90%

Dashboard Scans Assets Secinfo Configuration Extras Administration Help

Task: My First Scan

Name: My First Scan
Comment:
Target: test12
Alerts:
Schedule: (Next due: over)
Add to Assets: yes
Apply Overrides: yes
Min QoD: 70%
Alterable Task: no
Auto Delete Reports: Do not automatically delete reports
Scanner: OpenVAS Default (Type: OpenVAS Scanner)
Scan Config: Full and fast
Order for target hosts: Sequential
Network Source Interface:
Maximum concurrently executed NVTs per host: 4
Maximum concurrently scanned hosts: 20

Status: Done
Duration of last scan: 12 minutes 37 seconds
Average scan duration: 12 minutes 37 seconds
Reports: 1 (Finished: 1, Last: Jul 28 2024)
Results: 13
Notes: 0
Overrides: 0

Use Task (none)

ÖNEMLİ (Bulguların Teknik Analizi): Dashboard üzerinde gördüğümüz rakamlar sadece matematiksel ifadeler değildir; hedefin risk profilidir (CVSS skorlarına göre sınıflandırılmış High, Medium, Low bulgular). Taranan hedefin satırındaki zafiyet sayısına tıklayarak teknik detayların içine (Drill-down) girmeliyiz.

Applications Places System Greenbone Security Assistant — Mozilla Firefox

Greenbone Security Assi: x Add-ons Manager x +

https://127.0.0.1/omp?cmd=get_results&filter=severity>Error and task_id=dc4db8a9-f71e-4b55-8aee-a2cf4247da1e&overrides= 80%

Dashboard Scans Assets Secinfo Configuration Extras Administration Help

Results by Severity Class (Total: 13)

Medium 1
Low 2
Log 10

Results vulnerability word cloud

SSH Supported
Algorithms
Weak
Detection
Services
Protocol

Results by CVSS (Total: 13)

10 9 8 7 6 5 4 3 2 1 0
N/A 0 1 2 3 4 5 6 7 8 9 10

Vulnerability	Severity	QoD	Host	Location	Created
SSH Protocol Versions Supported	0.0 (Log)	95%	10.10.154.44	22/tcp	Sun Jul 28 09:33:59 2024
SSH Server type and version	0.0 (Log)	80%	10.10.154.44	22/tcp	Sun Jul 28 09:33:16 2024
ICMP Timestamp Detection	0.0 (Log)	80%	10.10.154.44	general/icmp	Sun Jul 28 09:36:07 2024
Services	0.0 (Log)	80%	10.10.154.44	8000/tcp	Sun Jul 28 09:33:11 2024
Services	0.0 (Log)	80%	10.10.154.44	22/tcp	Sun Jul 28 09:33:12 2024
No 404 check	0.0 (Log)	80%	10.10.154.44	8000/tcp	Sun Jul 28 09:35:49 2024
SSH Protocol Algorithms Supported	0.0 (Log)	80%	10.10.154.44	22/tcp	Sun Jul 28 09:33:55 2024
SSH Weak MAC Algorithms Supported	2.6 (Low)	95%	10.10.154.44	22/tcp	Sun Jul 28 09:35:56 2024
SSH Weak Encryption Algorithms Supported	4.3 (Medium)	95%	10.10.154.44	22/tcp	Sun Jul 28 09:36:00 2024
OS Detection Consolidation and Reporting	0.0 (Log)	80%	10.10.154.44	general/tcp	Sun Jul 28 09:35:50 2024

Listelenen zafiyetlerden birine tıkladığınızda OpenVAS sadece bir "açık var" demekle kalmaz; size zafiyetin tanımını, sistemdeki hangi yazılım versiyonundan kaynaklandığını ve en önemlisi **Remediation** (Çözüm/Yama) yöntemlerini sunar. Sektörel olarak bu veriler, sistem yöneticilerine iletmek üzere farklı formatlarda (PDF, CSV, XML) dışa aktarılabilir (Export). Bu raporlar, kurumun "Compliance" (Uyumluluk) kanıtları olarak saklanacak olan nihai güvenlik belgeleridir.